

Got you — from **OperatingSystem.pdf** those 3 topics are core OS concepts. Here's the breakdown:

1. Process

A **process** is a program in execution.

Key points:

- **State**: New → Ready → Running → Waiting → Terminated
- **Components**: Program code, Program counter, Stack, Data section, Heap
- **Process Control Block - PCB**: OS stores info about each process here. Includes PID, state, registers, memory info
- **Types**:
 - **Parent and Child process**: One process can create another using ``fork()``
 - **Independent vs Cooperative**: Some run alone, some need to share data

Why it matters: The OS scheduler decides which process gets CPU time. That's how you can run Chrome, Spotify, and VS Code at the same time.

2. Semaphore

A **semaphore** is a synchronization tool used to control access to shared resources by multiple processes.

Key points:

- Invented by Dijkstra
- It's basically an integer variable
- **Two operations**:
 - ``wait()`` / ``P()``: Decreases value. If value becomes negative, process waits
 - ``signal()`` / ``V()``: Increases value. Wakes up a waiting process
- **Types**:

- ***Binary Semaphore***: Value 0 or 1. Used like a lock/mutex
- ***Counting Semaphore***: Value can be >1 . Used for resources with multiple instances, like 5 printers

Example: In a blood bank system, a counting semaphore can limit access so only 3 staff can update blood stock at once.

Semaphore solves the ***critical section problem*** so two processes don't corrupt the same data.

3. Deadlock

Deadlock happens when 2 or more processes are stuck forever, each waiting for a resource the other holds.

Necessary Conditions - Coffman Conditions. All 4 must be true:

1. ***Mutual Exclusion***: At least one resource must be non-sharable
2. ***Hold and Wait***: Process holds resources and waits for more
3. ***No Preemption***: OS can't forcibly take resources
4. ***Circular Wait***: Circular chain of processes waiting

Example:

Process A holds lock on Donor Table and waits for Patient Table.

Process B holds lock on Patient Table and waits for Donor Table.

Both stuck.

How OS handles Deadlock:

1. ***Prevention***: Break one of the 4 conditions
2. ***Avoidance***: Use Banker's Algorithm to check if granting a request is safe
3. ***Detection***: Periodically check for cycles in resource allocation graph

4. *Recovery*: Kill processes or preempt resources

Relation between them:

Processes compete for resources → Semaphores are used to manage that competition
→ If managed badly, it can lead to Deadlock.

Want me to also:

1. *Draw diagrams* for process states and deadlock?
2. *Give code examples* of semaphore in Java/C?
3. *Make short notes + Q&A* for exam prep from this PDF?